

```
1  /**
2   * Phase 6.5 / 6.6 – Admin dashboard: raw data, alerts (with resolve), system logs.
3   * Adulteration alert: sound + blinking indicator when adulterated sample identified.
4   * Review: modal with per-parameter graphs (pH, sugar, citric, ascorbic).
5   */
6  import { useState, useEffect, useCallback, useMemo, useRef } from "react";
7  import { useLocation } from "react-router-dom";
8  import {
9    PieChart,
10    Pie,
11    Tooltip,
12    Legend,
13    ResponsiveContainer,
14    BarChart,
15    Bar,
16    LineChart,
17    Line,
18    ScatterChart,
19    Scatter,
20    XAxis,
21    YAxis,
22    CartesianGrid,
23    ReferenceLine,
24  } from "recharts";
25  import {
26    fetchReadings,
27    fetchPredictions,
28    fetchLogs,
29    fetchDaily,
30    fetchWeekly,
31    fetchMonthly,
32    fetchEsp32Live,
33    getApiErrorMessage,
34  } from "../api/client";
35  import jsPDF from "jspdf";
36  import * as XLSX from "xlsx";
37  import type {
38    AggregationResponse,
39    SensorReadingItem,
40    PredictionItem,
41    SystemLogItem,
42    Esp32LiveResponse,
43    RobotStage,
44  } from "../api/types";
45  import { format, getISOWeek, startOfWeek, endOfWeek, startOfMonth, endOfMonth, addWeeks } from...
46  import { formatConfidence, confidencePercent } from "../utils/formatConfidence";
47  import { resolveConfidence } from "../utils/computeConfidence";
48  import {
49    cleaningInitialState,
50    cleaningSensorLabels,
51    cleaningChartColors,
52    evaluateCleaningStatus,
53    type CleaningSensorState,
54  } from "../data/cleaningReference";
55
56  /** Max rows fetched for PDF/CSV export (June 30 validation = 200). */
57  const EXPORT_FETCH_LIMIT = 500;
58
59  function resolvePeriodRange(
60    periodType: "day" | "week" | "month",
61    selectedDate: string,
62    selectedYear: number,
63    selectedWeek: number,
64    selectedMonth: number,
65  ): { dateFrom: string; dateTo: string } {
66    if (periodType === "day") {
67      return { dateFrom: selectedDate, dateTo: selectedDate };
68    }
69  }
```

```

69     if (periodType === "week") {
70         const jan4 = new Date(selectedYear, 0, 4);
71         const mondayOfWeek1 = startOfWeek(jan4, { weekStartsOn: 1 });
72         const targetMonday = addWeeks(mondayOfWeek1, selectedWeek - 1);
73         return {
74             dateFrom: format(targetMonday, "yyyy-MM-dd"),
75             dateTo: format(endOfWeek(targetMonday, { weekStartsOn: 1 }), "yyyy-MM-dd"),
76         };
77     }
78     const start = startOfMonth(new Date(selectedYear, selectedMonth - 1));
79     const end = endOfMonth(new Date(selectedYear, selectedMonth - 1));
80     return {
81         dateFrom: format(start, "yyyy-MM-dd"),
82         dateTo: format(end, "yyyy-MM-dd"),
83     };
84 }
85
86 function getPeriodFileBase(prefix: string, periodType: "day" | "week" | "month", selectedDate: ...
87     if (periodType === "day") return `${prefix}-${selectedDate}`;
88     if (periodType === "week") return `${prefix}-${selectedYear}-W${selectedWeek}`;
89     return `${prefix}-${selectedYear}-${String(selectedMonth).padStart(2, "0")}`;
90 }
91
92 function downloadCsvFile(filename: string, rows: string[][]): void {
93     const csv = rows
94         .map((row) => row.map((cell) => `${String(cell).replace(/"/g, '"')}`").join(","))
95         .join("\n");
96     const blob = new Blob(["\ufeff", csv], { type: "text/csv;charset=utf-8;" });
97     const url = URL.createObjectURL(blob);
98     const link = document.createElement("a");
99     link.href = url;
100     link.download = filename;
101     link.click();
102     URL.revokeObjectURL(url);
103 }
104
105 function downloadExcelFile(filename: string, sheets: { name: string; rows: string[][] }[]): vo...
106     const wb = XLSX.utils.book_new();
107     for (const sheet of sheets) {
108         XLSX.utils.book_append_sheet(wb, XLSX.utils.aoa_to_sheet(sheet.rows), sheet.name);
109     }
110     XLSX.writeFile(wb, filename);
111 }
112
113 function buildAggregationSummaryRows(agg: AggregationResponse, periodLabel: string): string[][]...
114     return [
115         ["Coconut Water Quality Report"],
116         ["Period", periodLabel],
117         ["Period type", agg.period_type],
118         [],
119         ["Summary"],
120         ["Total readings", String(agg.count)],
121         ["Authentic", String(agg.authenticity.authentic)],
122         ["Adulterated", String(agg.authenticity.adulterated)],
123         ["Overall status", agg.status],
124         [],
125         ["Averages"],
126         ["pH", agg.averages.ph != null ? String(agg.averages.ph) : ""],
127         ["TDS", agg.averages.tds != null ? String(agg.averages.tds) : ""],
128         ["Temperature", agg.averages.temperature != null ? String(agg.averages.temperature) : ""],
129         ["Turbidity", agg.averages.turbidity != null ? String(agg.averages.turbidity) : ""],
130         ["Sugar %", agg.averages.predicted_sugar != null ? String(agg.averages.predicted_sugar) : ...
131         ["Citric %", agg.averages.predicted_citric != null ? String(agg.averages.predicted_citric)...
132         ["Ascorbic %", agg.averages.predicted_ascorbic != null ? String(agg.averages.predicted_asc...
133     ];
134 }
135
136 function sensorReadingExportRows(readings: SensorReadingItem[]): string[][] {

```

```

137     const headers = ["ID", "Date", "Time", "pH", "TDS", "Temperature", "Turbidity", "Device"];
138     const rows: string[][] = [headers];
139     for (const r of readings) {
140         const dt = new Date(r.timestamp);
141         rows.push([
142             String(r.id),
143             format(dt, "yyyy-MM-dd"),
144             format(dt, "HH:mm"),
145             String(r.ph),
146             String(r.tds),
147             String(r.temperature),
148             String(r.turbidity),
149             r.source_device_id ?? "",
150         ]);
151     }
152     return rows;
153 }
154
155 function predictionExportRows(predictions: PredictionItem[], readingById: Map<number, SensorRe...
156     const headers = ["ID", "Reading", "Date", "Time", "pH", "Sugar %", "Citric %", "Ascorbic %",...
157     const rows: string[][] = [headers];
158     for (const p of predictions) {
159         const r = readingById.get(p.reading);
160         const dt = new Date(p.timestamp);
161         rows.push([
162             String(p.id),
163             String(p.reading),
164             format(dt, "yyyy-MM-dd"),
165             format(dt, "HH:mm"),
166             r?.ph !== null ? String(r.ph) : "",
167             String(p.predicted_sugar),
168             String(p.predicted_citric),
169             String(p.predicted_ascorbic),
170             p.authenticity_status === "authentic" ? "Authentic" : "Adulterated",
171             p.confidence !== null ? confidencePercent(p.confidence) : "",
172         ]);
173     }
174     return rows;
175 }
176
177 /** Typical ranges for coconut water (for review comparison). */
178 const TYPICAL_RANGES = {
179     ph: { min: 4.5, max: 5.5 },
180     sugar: { min: 4, max: 6 },
181     citric: { min: 0.08, max: 0.15 },
182     ascorbic: { min: 0.05, max: 0.12 },
183 };
184
185 type ReadingData = {
186     ph: number | null;
187     citric: number | null;
188     ascorbic: number | null;
189     sugar: number | null;
190     authentic?: boolean;
191     confidence?: number | null;
192 };
193
194 type LiveReadingSlot = ReadingData & { tds?: number; temperature?: number; turbidity?: number ...
195
196 type LiveReadingsState = [LiveReadingSlot | null, LiveReadingSlot | null, LiveReadingSlot | nu...
197
198 function predictionToLiveSlot(pred: PredictionItem, reading: SensorReadingItem): LiveReadingSl...
199     return {
200         ph: reading.ph,
201         citric: pred.predicted_citric,
202         ascorbic: pred.predicted_ascorbic,
203         sugar: pred.predicted_sugar,
204         authentic: pred.authenticity_status === "authentic",

```

```

205         confidence: resolveConfidence(
206             pred.confidence,
207             pred.predicted_sugar,
208             pred.predicted_citric,
209             pred.predicted_ascorbic,
210             reading.ph,
211         ),
212         tds: reading.tds,
213         temperature: reading.temperature,
214         turbidity: reading.turbidity,
215     };
216 }
217
218 /** Sensor-only slot while ESP32 batch is still collecting (ML runs after 3rd reading). */
219 function bufferedToLiveSlot(raw: NonNullable<Esp32LiveResponse["buffered_readings"]>[number]):...
220     const sugar = raw.predicted_sugar ?? null;
221     const citric = raw.predicted_citric ?? null;
222     const ascorbic = raw.predicted_ascorbic ?? null;
223     return {
224         ph: raw.pH,
225         citric,
226         ascorbic,
227         sugar,
228         authentic:
229             raw.authenticity_status === "authentic"
230                 ? true
231                 : raw.authenticity_status === "adulterated"
232                 ? false
233                 : undefined,
234         confidence: resolveConfidence(undefined, sugar, citric, ascorbic, raw.pH),
235         tds: raw.tds,
236         temperature: raw.temperature,
237         turbidity: raw.turbidity,
238     };
239 }
240
241 /** Build First/Second/Third slots from esp32-live while batch is collecting. */
242 function buildBatchSlots(live: Esp32LiveResponse): LiveReadingsState {
243     const partial: LiveReadingsState = [null, null, null];
244     const buffered = live.buffered_readings ?? [];
245     for (let i = 0; i < live.buffered_count && i < 3; i++) {
246         let raw = buffered[i];
247         // Prefer buffer slot; fall back to live.current for any missing slot (esp. reading 1).
248         if (!raw && live.current) {
249             raw = {
250                 pH: live.current.pH,
251                 tds: live.current.tds,
252                 temperature: live.current.temperature,
253                 turbidity: live.current.turbidity,
254             };
255         }
256         if (raw) partial[i] = bufferedToLiveSlot(raw);
257     }
258     // Guarantee slot 0 while a batch is open and we have a current snapshot.
259     if (live.buffered_count >= 1 && !partial[0] && live.current) {
260         partial[0] = bufferedToLiveSlot({
261             pH: live.current.pH,
262             tds: live.current.tds,
263             temperature: live.current.temperature,
264             turbidity: live.current.turbidity,
265         });
266     }
267     return partial;
268 }
269
270 /** Finished 3-reading batch from esp32-live (includes ML + authenticity). */
271 function buildCompletedBatchSlots(
272     batch: NonNullable<Esp32LiveResponse["completed_batch"]>,

```

```

273 ): LiveReadingsState {
274   const slots: LiveReadingsState = [null, null, null];
275   batch.readings.slice(0, 3).forEach((raw, i) => {
276     slots[i] = bufferedToLiveSlot(raw);
277   });
278   return slots;
279 }
280
281 function completedBatchToSummary(
282   batch: NonNullable<Esp32LiveResponse["completed_batch"]>,
283 ): ReadingData {
284   const avg = batch.average;
285   return {
286     ph: avg.ph ?? null,
287     citric: avg.predicted_citric ?? null,
288     ascorbic: avg.predicted_ascorbic ?? null,
289     sugar: avg.predicted_sugar ?? null,
290     authentic:
291       avg.authenticity_status === "authentic"
292         ? true
293         : avg.authenticity_status === "adulterated"
294           ? false
295           : undefined,
296     confidence: avg.confidence ?? null,
297   };
298 }
299
300 function rawToLiveSlot(raw: {
301   pH: number;
302   tds: number;
303   temperature: number;
304   turbidity: number;
305 }): LiveReadingSlot {
306   return {
307     ph: raw.pH,
308     citric: null,
309     ascorbic: null,
310     sugar: null,
311     tds: raw.tds,
312     temperature: raw.temperature,
313     turbidity: raw.turbidity,
314   };
315 }
316
317 function averageReadings(readings: (ReadingData | null)[]): ReadingData {
318   const valid = readings.filter(
319     (x): x is ReadingData =>
320       x != null && x.ph != null && x.citric != null && x.ascorbic != null && x.sugar != null
321   );
322   if (valid.length === 0) return { ph: null, citric: null, ascorbic: null, sugar: null, confid...
323   const withConf = valid.filter((x) => x.confidence != null);
324   const avgSugar = valid.reduce((s, x) => s + (x.sugar ?? 0), 0) / valid.length;
325   const avgCitric = valid.reduce((s, x) => s + (x.citric ?? 0), 0) / valid.length;
326   const avgAscorbic = valid.reduce((s, x) => s + (x.ascorbic ?? 0), 0) / valid.length;
327   const avgPh = valid.reduce((s, x) => s + (x.ph ?? 0), 0) / valid.length;
328   return {
329     ph: avgPh,
330     citric: avgCitric,
331     ascorbic: avgAscorbic,
332     sugar: avgSugar,
333     confidence:
334       withConf.length > 0
335         ? withConf.reduce((s, x) => s + (x.confidence ?? 0), 0) / withConf.length
336         : resolveConfidence(null, avgSugar, avgCitric, avgAscorbic, avgPh),
337   };
338 }
339
340 const EMPTY_READING: ReadingData = { ph: null, citric: null, ascorbic: null, sugar: null };

```

```
341
342   type SensorSnapshot = {
343     pH: number | null;
344     tds: number;
345     temperature: number;
346     turbidity: number;
347   };
348
349   /** Home overview: show realistic same-sample demo values instead of raw ESP32 variance. */
350   const USE_HOME_DEMO_READINGS = false;
351
352   function randomInRange(min: number, max: number): number {
353     return min + Math.random() * (max - min);
354   }
355
356   /** Three close readings for one sample – third pH fixed at 5.00. */
357   function generateDemoReadingBatch(): [ReadingData, ReadingData, ReadingData] {
358     const base = {
359       citric: randomInRange(0.09, 0.14),
360       ascorbic: randomInRange(0.0002, 0.00032),
361       sugar: randomInRange(3.8, 5.2),
362     };
363     const jitter = (scale: number) => (Math.random() - 0.5) * scale;
364     const mk = (ph: number): ReadingData => ({
365       ph: +ph.toFixed(2),
366       citric: +(Math.max(0.04, base.citric + jitter(0.015))).toFixed(4),
367       ascorbic: +(Math.max(0.0001, base.ascorbic + jitter(0.00003))).toFixed(4),
368       sugar: +(Math.max(1.8, base.sugar + jitter(0.35))).toFixed(2),
369       authentic: true,
370     });
371     return [
372       mk(5.0 + jitter(0.08)),
373       mk(5.0 + jitter(0.08)),
374       mk(5.0),
375     ];
376   }
377
378   type HomeDemoBatch = {
379     first: ReadingData;
380     second: ReadingData;
381     third: ReadingData;
382     summary: ReadingData;
383     sensorSnapshot: SensorSnapshot;
384   };
385
386   function buildHomeDemoBatch(): HomeDemoBatch {
387     const [first, second, third] = generateDemoReadingBatch();
388     const summary = averageReadings([first, second, third]);
389     return {
390       first,
391       second,
392       third,
393       summary,
394       sensorSnapshot: {
395         pH: first.ph,
396         tds: Math.round(randomInRange(280, 380)),
397         temperature: +randomInRange(27, 29).toFixed(1),
398         turbidity: Math.round(randomInRange(20, 60)),
399       },
400     };
401   }
402
403   /** Colors for pH, citric acid, ascorbic acid, sugar (dots and charts). */
404   const PARAM_COLORS = { ph: "#4A90E2", citric: "#F5A623", ascorbic: "#14B8A6", sugar: "#27AE60"...
405
406   /** Matches firmware ENABLE_PH_SENSOR=1 – pH on D35 */
407   const PH_SENSOR_ENABLED = true;
408
```

```

409  /** Live ESP32 sensor panel – polls backend for TDS, temp, turbidity (pH when enabled). */
410  function SensorPredictPanel({
411    currentSnapshot,
412    bufferedCount,
413    requiredCount,
414    waiting,
415  }): {
416    currentSnapshot: SensorSnapshot | null;
417    bufferedCount: number;
418    requiredCount: number;
419    waiting: boolean;
420  } {
421    const sensors: { key: keyof SensorSnapshot; label: string; format: (v: number) => string }[] =
422      ...(PH_SENSOR_ENABLED
423        ? [{ key: "pH" as const, label: "pH", format: (v: number) => v.toFixed(2) }]
424        : []),
425      { key: "tds", label: "TDS", format: (v) => v.toFixed(1) },
426      { key: "temperature", label: "Temp °C", format: (v) => v.toFixed(1) },
427      { key: "turbidity", label: "Turbidity", format: (v) => v.toFixed(2) },
428    ];
429
430    return (
431      <div className="admin-sensor-predict-panel admin-report-panel">
432        <div className="admin-sensor-predict-header">
433          <h3 className="admin-report-panel-title">Sensor reading (ESP32 live)</h3>
434          {!PH_SENSOR_ENABLED && (
435            <span className="admin-sensor-predict-hint">pH sensor not connected</span>
436          )}
437        </div>
438
439        {waiting && !currentSnapshot && (
440          <p className="admin-sensor-predict-wait">Waiting for ESP32 measurement...</p>
441        )}
442
443        {bufferedCount > 0 && bufferedCount < requiredCount && (
444          <p className="admin-sensor-predict-buffer">
445            Receiving batch: {bufferedCount}/{requiredCount}
446          </p>
447        )}
448
449        <div className="admin-sensor-snapshot-grid">
450          {sensors.map(({ key, label, format }) => (
451            <div key={String(key)} className="admin-sensor-snapshot-cell">
452              <span className="admin-sensor-snapshot-label">{label}</span>
453              <span className="admin-sensor-snapshot-value">
454                {currentSnapshot && currentSnapshot[key] != null
455                  ? format(currentSnapshot[key] as number)
456                  : "-"}
457              </span>
458            </div>
459          ))}
460        </div>
461      </div>
462    );
463  }
464
465  /** Period selector for first page: Day / Week / Month with date selection. */
466  function PeriodSelectorBar({
467    periodType,
468    setPeriodType,
469    selectedDate,
470    setSelectedDate,
471    selectedYear,
472    setSelectedYear,
473    selectedWeek,
474    setSelectedWeek,
475    selectedMonth,
476    setSelectedMonth,

```

```

477     onApply,
478     onDownloadPdf,
479     onDownloadCsv,
480     onDownloadExcel,
481     loading,
482     variant = "dark",
483   }: {
484     periodType: "day" | "week" | "month";
485     setPeriodType: (v: "day" | "week" | "month") => void;
486     selectedDate: string;
487     setSelectedDate: (v: string) => void;
488     selectedYear: number;
489     setSelectedYear: (v: number) => void;
490     selectedWeek: number;
491     setSelectedWeek: (v: number) => void;
492     selectedMonth: number;
493     setSelectedMonth: (v: number) => void;
494     onApply: () => void;
495     onDownloadPdf: () => void;
496     onDownloadCsv: () => void;
497     onDownloadExcel: () => void;
498     loading: boolean;
499     variant?: "dark" | "light";
500   }) {
501     return (
502       <div className={`admin-report-period-bar admin-report-period-bar--${variant}`}>
503         <label>
504           Period:{" "}
505           <select value={periodType} onChange={(e) => setPeriodType(e.target.value as "day" | "w...
506             <option value="day">Day</option>
507             <option value="week">Week</option>
508             <option value="month">Month</option>
509           </select>
510         </label>
511         {periodType === "day" && (
512           <label>
513             Date: <input type="date" value={selectedDate} onChange={(e) => setSelectedDate(e.tar...
514           </label>
515         )}
516         {periodType === "week" && (
517           <>
518             <label>
519               Year: <input type="number" min={2020} max={2030} value={selectedYear} onChange={(e...
520             </label>
521             <label>
522               Week: <input type="number" min={1} max={53} value={selectedWeek} onChange={(e) => ...
523             </label>
524           </>
525         )}
526         {periodType === "month" && (
527           <>
528             <label>
529               Year: <input type="number" min={2020} max={2030} value={selectedYear} onChange={(e...
530             </label>
531             <label>
532               Month: <input type="number" min={1} max={12} value={selectedMonth} onChange={(e) =...
533             </label>
534           </>
535         )}
536       <button type="button" onClick={onApply} disabled={loading}>
537         {loading ? "Loading..." : "Apply"}
538       </button>
539       <span className="admin-download-group">
540         <button type="button" onClick={onDownloadPdf} disabled={loading} className="admin-down...
541         PDF
542       </button>
543         <button type="button" onClick={onDownloadCsv} disabled={loading} className="admin-down...
544         CSV

```

```

545         </button>
546         <button type="button" onClick={onDownloadExcel} disabled={loading} className="admin-do...
547             Excel
548         </button>
549     </span>
550 </div>
551 );
552 }
553
554 /** Production report layout: Assembly Lines style (dark theme, Summary + First/Second/Third p...
555 function ProductionReportOverview({
556     readings,
557     liveReadings,
558     activeReadingIndex,
559     bufferedCount,
560     requiredCount,
561     homeDemo,
562     robotStage,
563     completedVerdict,
564 }): {
565     readings: SensorReadingItem[];
566     liveReadings: LiveReadingsState;
567     activeReadingIndex: number;
568     bufferedCount: number;
569     requiredCount: number;
570     homeDemo: HomeDemoBatch | null;
571     robotStage: RobotStage | null;
572     completedVerdict: "authentic" | "adulterated" | null;
573 }) {
574     const hasLiveSession = liveReadings.some((r) => r !== null);
575     const isCollecting = bufferedCount > 0 && bufferedCount < requiredCount;
576
577     const { summary, first, second, third } = useMemo(() => {
578         if (homeDemo) {
579             const showFirst = !isCollecting || bufferedCount >= 1;
580             const showSecond = !isCollecting || bufferedCount >= 2;
581             const showThird = !isCollecting || bufferedCount >= 3;
582             const filled = [
583                 showFirst ? homeDemo.first : null,
584                 showSecond ? homeDemo.second : null,
585                 showThird ? homeDemo.third : null,
586             ].filter((s): s is ReadingData => s !== null && s.ph !== null);
587             return {
588                 first: showFirst ? homeDemo.first : EMPTY_READING,
589                 second: showSecond ? homeDemo.second : EMPTY_READING,
590                 third: showThird ? homeDemo.third : EMPTY_READING,
591                 summary: filled.length === 3 ? homeDemo.summary : EMPTY_READING,
592             };
593         }
594
595         if (isCollecting) {
596             const firstSlot = bufferedCount >= 1 && liveReadings[0] ? liveReadings[0] : EMPTY_READIN...
597             const secondSlot = bufferedCount >= 2 && liveReadings[1] ? liveReadings[1] : EMPTY_READI...
598             const thirdSlot = EMPTY_READING;
599             return {
600                 first: firstSlot,
601                 second: secondSlot,
602                 third: thirdSlot,
603                 summary: averageReadings([
604                     bufferedCount >= 1 ? firstSlot : null,
605                     bufferedCount >= 2 ? secondSlot : null,
606                     null,
607                 ]),
608             };
609         }
610
611         if (hasLiveSession) {
612             return {

```

```

613         first: liveReadings[0] ?? EMPTY_READING,
614         second: liveReadings[1] ?? EMPTY_READING,
615         third: liveReadings[2] ?? EMPTY_READING,
616         summary: averageReadings(liveReadings),
617     };
618 }
619
620 // No live ESP32 session yet – keep panels empty (do not show old DB / sample data).
621 return {
622     summary: EMPTY_READING,
623     first: EMPTY_READING,
624     second: EMPTY_READING,
625     third: EMPTY_READING,
626 };
627 }, [liveReadings, hasLiveSession, isCollecting, bufferedCount, requiredCount, homeDemo]);
628
629 const fmt = (v: number | null) => (v !== null ? (v < 0.01 ? v.toFixed(4) : v.toFixed(2)) : "\...
630
631 const MetricRow = ({ label, value, color }: { label: string; value: number | null; color: st...
632     <div className="admin-report-metric-row">
633         <span className="admin-report-metric-dot" style={{ backgroundColor: color }} aria-hidden...
634         <span className="admin-report-metric-label">{label}</span>
635         <span className="admin-report-metric-value">{fmt(value)}</span>
636     </div>
637 );
638
639 const ReadingPanel = ({ title, data }: { title: string; data: ReadingData }) => (
640     <div className="admin-report-panel admin-report-panel-compact">
641         <h3 className="admin-report-panel-title">{title}</h3>
642         <MetricRow label="pH" value={data.ph} color={PARAM_COLORS.ph} />
643         <MetricRow label="Citric acid %" value={data.citric} color={PARAM_COLORS.citric} />
644         <MetricRow label="Ascorbic acid %" value={data.ascorbic} color={PARAM_COLORS.ascorbic} />
645         <MetricRow label="Sugar %" value={data.sugar} color={PARAM_COLORS.sugar} />
646     </div>
647 );
648
649 const paramsPieData = useMemo(() => {
650     const s = summary;
651     if (s.ph == null && s.citric == null && s.ascorbic == null && s.sugar == null) return [];
652     const v = (x: number | null, max: number) => (x !== null ? Math.max(0.5, Math.min(100, (x /...
653     return [
654         { name: "pH", value: v(s.ph, 7), raw: s.ph, fill: PARAM_COLORS.ph },
655         { name: "Citric %", value: v(s.citric ?? 0, 0.2), raw: s.citric ?? 0, fill: PARAM_COLORS...
656         { name: "Ascorbic %", value: v(s.ascorbic ?? 0, 0.15), raw: s.ascorbic ?? 0, fill: PARAM...
657         { name: "Sugar %", value: v(s.sugar ?? 0, 10), raw: s.sugar ?? 0, fill: PARAM_COLORS.sug...
658     ];
659 }, [summary]);
660
661 const latestSensorReading = useMemo(() => {
662     const latestLive = liveReadings[2] ?? liveReadings[1] ?? liveReadings[0];
663     if (latestLive?.tds !== null && latestLive.temperature !== null && latestLive.turbidity !== n...
664     return {
665         ph: latestLive.ph ?? 0,
666         tds: latestLive.tds,
667         turbidity: latestLive.turbidity,
668         temperature: latestLive.temperature,
669     };
670 }
671 if (readings.length === 0) return null;
672 return [...readings].sort(
673     (a, b) => new Date(b.timestamp).getTime() - new Date(a.timestamp).getTime()
674 )[0];
675 }, [readings, liveReadings]);
676
677 const currentCleaning: CleaningSensorState | null = latestSensorReading
678 ? {
679     ph: latestSensorReading.ph,
680     tds: latestSensorReading.tds,

```

```

681         turbidity: latestSensorReading.turbidity,
682         temperature: latestSensorReading.temperature,
683     }
684     : null;
685
686     const cleaningEvaluation = useMemo(() => {
687         if (!currentCleaning) return null;
688         return evaluateCleaningStatus(currentCleaning);
689     }, [currentCleaning]);
690
691     const cleaningChartData = useMemo(() => {
692         const keys = Object.keys(cleaningInitialState) as (keyof CleaningSensorState)[];
693         const ranges: Record<keyof CleaningSensorState, { min: number; max: number }> = {
694             ph: { min: 6.5, max: 7.5 },
695             tds: { min: 0, max: 200 },
696             turbidity: { min: 0, max: 10 },
697             temperature: { min: 20, max: 30 },
698         };
699         const norm = (v: number, min: number, max: number) =>
700             Math.max(0, Math.min(100, ((v - min) / (max - min || 0.001)) * 100));
701         const perSensor = cleaningEvaluation?.perSensor;
702         const hasReading = currentCleaning !== null;
703         return keys.map((key) => ({
704             name: cleaningSensorLabels[key],
705             initial: norm(cleaningInitialState[key], ranges[key].min, ranges[key].max),
706             current: hasReading
707                 ? norm(currentCleaning[key], ranges[key].min, ranges[key].max)
708                 : 4,
709             initialRaw: cleaningInitialState[key],
710             currentRaw: currentCleaning?.[key] ?? null,
711             hasReading,
712             matches: hasReading ? (perSensor?.[key] ?? false) : false,
713         }));
714     }, [currentCleaning, cleaningEvaluation]);
715
716     const cleaningChartVerdict = cleaningEvaluation
717         ? cleaningEvaluation.finished
718           ? "Cleaning finished"
719           : "Clean again"
720         : "Awaiting sensor reading";
721
722     const robotStages: Array<{ stage: RobotStage; label: string }> = [
723         { stage: "initial_position", label: "Initial Position" },
724         { stage: "detection_station", label: "Detection Station" },
725         { stage: "cleaning_station", label: "Cleaning Station" },
726         { stage: "drying_station", label: "Drying Station" },
727     ];
728
729     const stepCircleClass = (step: 1 | 2 | 3) => {
730         if (!hasLiveSession && !isCollecting) return "";
731         const completed = isCollecting ? step <= bufferedCount : liveReadings[step - 1] !== null;
732         if (activeReadingIndex === step) return "admin-report-step-circle-active";
733         if (completed) return "admin-report-step-circle-done";
734         return "";
735     };
736
737     return (
738         <div className="admin-report-dashboard">
739             <header className="admin-report-header">
740                 <h1 className="admin-report-title admin-report-title-white">Coconut Water Authenticity...
741                 {completedVerdict && (
742                     <p
743                         className={`admin-authenticity-verdict admin-authenticity-verdict-${completedVerdi...
744                         role="status"
745                     >
746                         Result: {completedVerdict === "authentic" ? "Authentic" : "Adulterated"}
747                     </p>
748                 )}

```

```

749     </header>
750
751     <div className="admin-report-panels-wrap">
752       <div className="admin-report-step-indicator">
753         <div className="admin-report-step-item">
754           <div className={`admin-report-step-circle ${stepCircleClass(1)}`}>1</div>
755           <span className="admin-report-step-label">First reading</span>
756         </div>
757         <div className="admin-report-step-connector" />
758         <div className="admin-report-step-item">
759           <div className={`admin-report-step-circle ${stepCircleClass(2)}`}>2</div>
760           <span className="admin-report-step-label">Second reading</span>
761         </div>
762         <div className="admin-report-step-connector" />
763         <div className="admin-report-step-item">
764           <div className={`admin-report-step-circle ${stepCircleClass(3)}`}>3</div>
765           <span className="admin-report-step-label">Third reading</span>
766         </div>
767       </div>
768       <div className="admin-report-panels-row">
769         <ReadingPanel title="First reading" data={first} />
770         <ReadingPanel title="Second reading" data={second} />
771         <ReadingPanel title="Third reading" data={third} />
772         <div className="admin-report-panel admin-report-panel-compact admin-report-summary">
773           <h3 className="admin-report-panel-title admin-report-summary-title">Average readin...
774           <MetricRow label="pH" value={summary.ph} color={PARAM_COLORS.ph} />
775           <MetricRow label="Citric acid %" value={summary.citric} color={PARAM_COLORS.citric...
776           <MetricRow label="Ascorbic acid %" value={summary.ascorbic} color={PARAM_COLORS.as...
777           <MetricRow label="Sugar %" value={summary.sugar} color={PARAM_COLORS.sugar} />
778         </div>
779       </div>
780     </div>
781
782     <div className="admin-report-charts-row">
783       <div className="admin-report-panel admin-report-chart-panel">
784         <h3 className="admin-report-panel-title">Parameters (pH, citric, ascorbic, sugar)</h...
785         <div className="admin-chart-container" style={{ overflow: "visible" }}>
786           {paramsPieData.length > 0 ? (
787             <ResponsiveContainer width="100%" height="100%">
788               <PieChart margin={{ top: 8, right: 8, left: 8, bottom: 48 }}>
789                 <Pie
790                   data={paramsPieData}
791                   cx="50%"
792                   cy="42%"
793                   innerRadius={55}
794                   outerRadius={85}
795                   paddingAngle={2}
796                   dataKey="value"
797                   label={false}
798                 />
799                 <Tooltip
800                   contentStyle={{ background: "#1e293b", border: "1px solid #475569" }}
801                   formatter={({_v: number, _name: string, item: { payload?: { raw?: number; n...
802                     const raw = item.payload?.raw;
803                     const label = item.payload?.name ?? "";
804                     if (raw == null) return ["-", label];
805                     const display = label === "pH" ? raw.toFixed(2) : raw < 0.01 ? raw.toFix...
806                     return [display, label];
807                   }}
808                 />
809                 <Legend
810                   verticalAlign="bottom"
811                   layout="horizontal"
812                   wrapperStyle={{ color: "#fff", paddingTop: "8px" }}
813                 />
814               </PieChart>
815             </ResponsiveContainer>
816           ) : (

```

```

817         <div style={{ display: "flex", alignItems: "center", justifyContent: "center", h...
818             No data – values will appear when readings are available
819         </div>
820     )}
821 </div>
822 </div>
823 <div className="admin-report-panel admin-report-chart-panel admin-robot-stage-panel">
824     <h3 className="admin-report-panel-title">Robot Position</h3>
825     <div className="admin-robot-stage-list" role="status" aria-live="polite">
826         {robotStages.map(({ stage, label }) => {
827             const active = robotStage === stage;
828             const isDetection = stage === "detection_station";
829             return (
830                 <div
831                     key={stage}
832                     className={`admin-robot-stage-item ${active ? "admin-robot-stage-item-active...
833                 >
834                 <span
835                     className={[
836                         "admin-robot-stage-light",
837                         active ? "admin-robot-stage-light-active" : "",
838                         active && isDetection ? "admin-robot-stage-light-blink" : "",
839                     ]
840                         .filter(Boolean)
841                         .join(" ")}
842                     aria-hidden
843                 />
844                 <span>{label}</span>
845             </div>
846         );
847     )}
848 </div>
849     {!robotStage && <p className="admin-robot-stage-waiting">Waiting for robot status...</...
850 </div>
851 <div className="admin-report-panel admin-report-chart-panel">
852     <h3 className="admin-report-panel-title">Cleaning verification</h3>
853     <div className="admin-chart-container">
854         <ResponsiveContainer width="100%" height="100%">
855             <BarChart data={cleaningChartData} margin={{ top: 8, right: 12, left: 8, bottom:...
856                 <CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.12)" vertical=...
857                 <XAxis dataKey="name" stroke="#fff" tick={{ fill: "#fff", fontSize: 11 }} inte...
858                 <YAxis stroke="#fff" tick={{ fill: "#fff" }} domain={[0, 100]} />
859                 <Tooltip
860                     contentStyle={{ background: "#1e293b", border: "1px solid #475569" }}
861                     labelFormatter={() => cleaningChartVerdict}
862                     formatter={({
863                         v: number,
864                         name: string,
865                         item: {
866                             payload?: {
867                                 initialRaw?: number;
868                                 currentRaw?: number | null;
869                                 hasReading?: boolean;
870                                 matches?: boolean;
871                             };
872                         }
873                     }) => {
874                         const p = item.payload;
875                         if (name === "Permanent reading") {
876                             const raw = p?.initialRaw;
877                             return [raw != null ? raw.toFixed(2) : String(v), name];
878                         }
879                         if (!p?.hasReading) return ["No reading", "Current reading"];
880                         const raw = p?.currentRaw;
881                         const display = raw != null ? raw.toFixed(2) : String(v);
882                         const suffix = p?.matches ? " (match)" : " (mismatch)";
883                         return [`${display}${suffix}`, name];
884                     }

```

```

885         />
886         <Legend verticalAlign="bottom" layout="horizontal" wrapperStyle={{ color: "#ff...
887         <Bar
888             dataKey="initial"
889             name="Permanent reading"
890             fill={cleaningChartColors.permanent}
891             radius={[4, 4, 0, 0]}
892         />
893         <Bar
894             dataKey="current"
895             name="Current reading"
896             fill={cleaningChartColors.current}
897             radius={[4, 4, 0, 0]}
898         />
899     </BarChart>
900 </ResponsiveContainer>
901 </div>
902 </div>
903 </div>
904
905 </div>
906 );
907 }
908
909 export default function AdminDashboard() {
910     const VALIDATION_DEMO_DATE = "2026-06-30";
911     const today = format(new Date(), "yyyy-MM-dd");
912     const [dateFrom, setDateFrom] = useState(today);
913     const [dateTo, setDateTo] = useState(today);
914     const [periodType, setPeriodType] = useState<"day" | "week" | "month">("day");
915     const [selectedDate, setSelectedDate] = useState(today);
916     const [selectedYear, setSelectedYear] = useState(new Date().getFullYear());
917     const [selectedWeek, setSelectedWeek] = useState(getISOWeek(new Date()));
918     const [selectedMonth, setSelectedMonth] = useState(new Date().getMonth() + 1);
919     const [statusFilter] = useState<string>("");
920     const [readings, setReadings] = useState<SensorReadingItem[]>([]);
921     const [liveReadings, setLiveReadings] = useState<LiveReadingsState>([null, null, null]);
922     const [liveSnapshot, setLiveSnapshot] = useState<SensorSnapshot | null>(null);
923     const [robotStage, setRobotStage] = useState<RobotStage | null>(null);
924     const [completedVerdict, setCompletedVerdict] = useState<"authentic" | "adulterated" | null>...
925     const [esp32Buffer, setEsp32Buffer] = useState({ count: 0, required: 3 });
926     const [demoSeed, setDemoSeed] = useState(0);
927     const homeDemo = useMemo(
928         () => (USE_HOME_DEMO_READINGS ? buildHomeDemoBatch() : null),
929         [demoSeed],
930     );
931     const lastBatchMaxPredId = useRef(0);
932     const prevBufferedCount = useRef(0);
933     const liveFilledCount = liveReadings.filter((r) => r !== null).length;
934     const isCollectingBatch = esp32Buffer.count > 0 && esp32Buffer.count < esp32Buffer.required;
935     const sessionComplete = !isCollectingBatch && liveFilledCount >= esp32Buffer.required;
936
937     const [predictions, setPredictions] = useState<PredictionItem[]>([]);
938     const [logs, setLogs] = useState<SystemLogItem[]>([]);
939     const [loadingReadings, setLoadingReadings] = useState(false);
940     const [loadingPredictions, setLoadingPredictions] = useState(false);
941     const [loadingLogs, setLoadingLogs] = useState(false);
942     const [error, setError] = useState<string | null>(null);
943     const [logLevelFilter, setLogLevelFilter] = useState("");
944     const [logComponentFilter, setLogComponentFilter] = useState("");
945     const [logSearchFilter, setLogSearchFilter] = useState("");
946
947     const loadReadings = useCallback(async () => {
948         setLoadingReadings(true);
949         setError(null);
950         try {
951             const data = await fetchReadings({
952                 date_from: dateFrom || undefined,

```

```
953         date_to: dateTo || undefined,
954         limit: EXPORT_FETCH_LIMIT,
955     });
956     setReadings(data);
957 } catch (err) {
958     setError(getApiErrorMessage(err));
959 } finally {
960     setLoadingReadings(false);
961 }
962 }, [dateFrom, dateTo]);
963
964 const loadPredictions = useCallback(async () => {
965     setLoadingPredictions(true);
966     setError(null);
967     try {
968         const data = await fetchPredictions({
969             date_from: dateFrom || undefined,
970             date_to: dateTo || undefined,
971             status: statusFilter || undefined,
972             limit: EXPORT_FETCH_LIMIT,
973         });
974         setPredictions(data);
975     } catch (err) {
976         setError(getApiErrorMessage(err));
977     } finally {
978         setLoadingPredictions(false);
979     }
980 }, [dateFrom, dateTo, statusFilter]);
981
982 const loadLogs = useCallback(async () => {
983     setLoadingLogs(true);
984     setError(null);
985     try {
986         const data = await fetchLogs({
987             date_from: dateFrom || undefined,
988             date_to: dateTo || undefined,
989             level: logLevelFilter || undefined,
990             component: logComponentFilter || undefined,
991             search: logSearchFilter || undefined,
992             limit: EXPORT_FETCH_LIMIT,
993         });
994         setLogs(data);
995     } catch (err) {
996         setError(getApiErrorMessage(err));
997     } finally {
998         setLoadingLogs(false);
999     }
1000 }, [dateFrom, dateTo, logLevelFilter, logComponentFilter, logSearchFilter]);
1001
1002 const loadAll = useCallback(() => {
1003     loadReadings();
1004     loadPredictions();
1005     loadLogs();
1006 }, [loadReadings, loadPredictions, loadLogs]);
1007
1008 const handleApplyPeriod = useCallback(() => {
1009     if (periodType === "day") {
1010         setDateFrom(selectedDate);
1011         setDateTo(selectedDate);
1012     } else if (periodType === "week") {
1013         const jan4 = new Date(selectedYear, 0, 4);
1014         const mondayOfWeek1 = startOfWeek(jan4, { weekStartsOn: 1 });
1015         const targetMonday = addWeeks(mondayOfWeek1, selectedWeek - 1);
1016         const start = targetMonday;
1017         const end = endOfWeek(targetMonday, { weekStartsOn: 1 });
1018         setDateFrom(format(start, "yyyy-MM-dd"));
1019         setDateTo(format(end, "yyyy-MM-dd"));
1020     } else {
```

```

1021     const start = startOfMonth(new Date(selectedYear, selectedMonth - 1));
1022     const end = endOfMonth(new Date(selectedYear, selectedMonth - 1));
1023     setDateFrom(format(start, "yyyy-MM-dd"));
1024     setDateTo(format(end, "yyyy-MM-dd"));
1025   }
1026   }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth]);
1027
1028   const handleGraphDateChange = useCallback((date: string) => {
1029     if (!date) return;
1030     setSelectedDate(date);
1031     setPeriodType("day");
1032     setDateFrom(date);
1033     setDateTo(date);
1034   }, []);
1035
1036   useEffect(() => {
1037     loadAll();
1038     // eslint-disable-next-line react-hooks/exhaustive-deps
1039   }, [dateFrom, dateTo]);
1040
1041   /** Poll ESP32 live sensor values + latest ML batch for overview panels. */
1042   useEffect(() => {
1043     let cancelled = false;
1044
1045     const pollEsp32Live = async () => {
1046       try {
1047         const live = await fetchEsp32Live();
1048         if (cancelled) return;
1049
1050         const required = live.required_count || 3;
1051         const collecting = live.buffered_count > 0 && live.buffered_count < required;
1052         const justCompleted =
1053           prevBufferedCount.current > 0 && live.buffered_count === 0;
1054         const recentSaved =
1055           !!live.latest_saved?.timestamp &&
1056           Date.now() - Date.parse(live.latest_saved.timestamp) <= 120_000;
1057
1058         const rs = live.robot_status;
1059         let stage: RobotStage | null = rs?.stage ?? null;
1060         if (rs?.updated_at) {
1061           const ageMs = Date.now() - Date.parse(rs.updated_at);
1062           // Match backend ROBOT_STATUS_STALE_SECONDS (180s).
1063           if (!Number.isFinite(ageMs) || ageMs > 180_000) {
1064             stage = null;
1065           }
1066         } else if (!rs) {
1067           stage = null;
1068         }
1069         setRobotStage(stage);
1070         setEsp32Buffer({ count: live.buffered_count, required });
1071
1072         // Finished batch from public esp32-live (no auth) – readings, graph, authenticity.
1073         if (live.completed_batch?.readings?.length) {
1074           const slots = buildCompletedBatchSlots(live.completed_batch);
1075           setLiveReadings(slots);
1076           const summary = completedBatchToSummary(live.completed_batch);
1077           const status = live.completed_batch.average?.authenticity_status;
1078           setCompletedVerdict(
1079             status === "authentic" ? "authentic" : status === "adulterated" ? "adulterated" : ...
1080           );
1081           if (live.current) {
1082             setLiveSnapshot({
1083               pH: PH_SENSOR_ENABLED ? live.current.pH : null,
1084               tds: live.current.tds,
1085               temperature: live.current.temperature,
1086               turbidity: live.current.turbidity,
1087             });
1088           }
1089         }
1090       } catch {
1091         // ignore
1092       }
1093     };
1094     pollEsp32Live();
1095   }, []);

```

```

1089     prevBufferedCount.current = live.buffered_count;
1090     return;
1091 }
1092
1093 // Idle / cleared buffer: wipe stale panels unless a batch just finished or was saved ...
1094 if (live.buffered_count === 0 && !justCompleted && !recentSaved) {
1095     setLiveReadings([null, null, null]);
1096     setLiveSnapshot(null);
1097     setCompletedVerdict(null);
1098     prevBufferedCount.current = 0;
1099     return;
1100 }
1101
1102 // New batch started – bump demo seed only; do NOT clear readings here
1103 // (clearing raced with buildBatchSlots and blanked Reading 1).
1104 if (live.buffered_count === 1 && prevBufferedCount.current === 0) {
1105     setDemoSeed((s) => s + 1);
1106 }
1107 prevBufferedCount.current = live.buffered_count;
1108
1109 // Sensor strip during active session or for a batch saved in the last 2 minutes.
1110 if (live.current && (live.buffered_count > 0 || justCompleted || recentSaved)) {
1111     setLiveSnapshot({
1112         pH: PH_SENSOR_ENABLED ? live.current.pH : null,
1113         tds: live.current.tds,
1114         temperature: live.current.temperature,
1115         turbidity: live.current.turbidity,
1116     });
1117 } else if (live.buffered_count === 0 && !live.current) {
1118     setLiveSnapshot(null);
1119 }
1120
1121 // 1 / 3 !' First column, 2 / 3 !' Second column
1122 if (collecting || live.buffered_count > 0) {
1123     setLiveReadings(buildBatchSlots(live));
1124     if (collecting) return;
1125 }
1126
1127 // Load completed ML batch after 3rd upload or when reopening dashboard with a fresh s...
1128 if (!justCompleted && !recentSaved) return;
1129
1130 const [preds, sensorRows] = await Promise.all([
1131     fetchPredictions({ limit: required }),
1132     fetchReadings({ limit: required }),
1133 ]);
1134 if (cancelled || preds.length < required) {
1135     setLiveReadings([null, null, null]);
1136     setLiveSnapshot(null);
1137     return;
1138 }
1139
1140 // Ignore stale DB batches (e.g. after clearing a test buffer).
1141 const ageMs = Date.now() - new Date(preds[0].timestamp).getTime();
1142 if (Number.isFinite(ageMs) && ageMs > 120_000) {
1143     setLiveReadings([null, null, null]);
1144     setLiveSnapshot(null);
1145     return;
1146 }
1147
1148 const batchTs = preds[0].timestamp.slice(0, 19);
1149 const batch = preds.filter((p) => p.timestamp.slice(0, 19) === batchTs);
1150 if (batch.length < required) return;
1151
1152 const maxId = Math.max(...batch.map((p) => p.id));
1153 if (maxId <= lastBatchMaxPredId.current) return;
1154 lastBatchMaxPredId.current = maxId;
1155
1156 const readingMap = new Map(sensorRows.map((r) => [r.id, r]));

```

```

1157         const ordered = [...batch].reverse();
1158         const slots: LiveReadingsState = [
1159             readingMap.has(ordered[0].reading)
1160                 ? predictionToLiveSlot(ordered[0], readingMap.get(ordered[0].reading!))
1161                 : null,
1162             ordered[1] && readingMap.has(ordered[1].reading)
1163                 ? predictionToLiveSlot(ordered[1], readingMap.get(ordered[1].reading!))
1164                 : null,
1165             ordered[2] && readingMap.has(ordered[2].reading)
1166                 ? predictionToLiveSlot(ordered[2], readingMap.get(ordered[2].reading!))
1167                 : null,
1168         ];
1169         setLiveReadings(slots);
1170         setPredictions((prev) => {
1171             const merged = [...preds, ...prev.filter((p) => !preds.some((x) => x.id === p.id))];
1172             return merged.slice(0, 100);
1173         });
1174         setReadings((prev) => {
1175             const merged = [...sensorRows, ...prev.filter((r) => !sensorRows.some((x) => x.id === r.id))];
1176             return merged.slice(0, 100);
1177         });
1178     } catch {
1179         /* live poll is best-effort */
1180     }
1181 };
1182
1183 pollEsp32Live();
1184 const timer = setInterval(pollEsp32Live, 2500);
1185 return () => {
1186     cancelled = true;
1187     clearInterval(timer);
1188 };
1189 }, []));
1190
1191 const handleDownloadPDF = useCallback(async () => {
1192     try {
1193         let agg: Awaited<ReturnType<typeof fetchDaily>>;
1194         let filename: string;
1195         if (periodType === "day") {
1196             agg = await fetchDaily(selectedDate);
1197             filename = `sensor-readings-${selectedDate}.pdf`;
1198         } else if (periodType === "week") {
1199             agg = await fetchWeekly(selectedYear, selectedWeek);
1200             filename = `sensor-readings-${selectedYear}-W${selectedWeek}.pdf`;
1201         } else {
1202             agg = await fetchMonthly(selectedYear, selectedMonth);
1203             filename = `sensor-readings-${selectedYear}-${String(selectedMonth).padStart(2, "0")}.pdf`;
1204         }
1205
1206         const pdf = new jsPDF("portrait", "mm", "a4");
1207         const pageW = pdf.internal.pageSize.getWidth();
1208         let y = 15;
1209
1210         pdf.setFontSize(16);
1211         const title = "Coconut Water Quality Report";
1212         const titleX = (pageW - pdf.getTextWidth(title)) / 2;
1213         pdf.text(title, titleX, y);
1214         y += 10;
1215
1216         pdf.setFontSize(10);
1217         pdf.text(`Period: ${agg.period}`, 14, y);
1218         y += 6;
1219         pdf.text(`Readings: ${agg.count} | Authentic: ${agg.authenticity.authentic} | Adulterate...`, 14, y);
1220         y += 10;
1221
1222         pdf.text("Averages:", 14, y);
1223         y += 6;
1224         pdf.text(`pH: ${agg.averages.ph ?? "-"} | TDS: ${agg.averages.tds ?? "-"} | Temp: ${agg.averages.temp ?? "-"}`, 14, y);

```

```

1225     y += 6;
1226     pdf.text(`Sugar %: ${agg.averages.predicted_sugar ?? "-"} | Citric %: ${agg.averages.pre...
1227     y += 12;
1228
1229     if (readings.length > 0) {
1230         const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
1231             periodType,
1232             selectedDate,
1233             selectedYear,
1234             selectedWeek,
1235             selectedMonth,
1236         );
1237         const exportReadings = await fetchReadings({
1238             date_from: exportFrom,
1239             date_to: exportTo,
1240             limit: EXPORT_FETCH_LIMIT,
1241         });
1242
1243         pdf.text(`Sensor readings (${exportReadings.length}):`, 14, y);
1244         y += 6;
1245         const colW = (pageW - 28) / 7;
1246         const headers = ["ID", "Date", "Time", "pH", "TDS", "Temp", "Turb"];
1247         pdf.setFont("helvetica", "bold");
1248         headers.forEach((h, i) => pdf.text(h, 14 + i * colW, y));
1249         pdf.setFont("helvetica", "normal");
1250         y += 6;
1251         for (const r of exportReadings) {
1252             if (y > 270) {
1253                 pdf.addPage();
1254                 y = 15;
1255             }
1256             const dt = new Date(r.timestamp);
1257             const dateStr = format(dt, "yyyy-MM-dd");
1258             const timeStr = format(dt, "HH:mm");
1259             const row = [String(r.id), dateStr, timeStr, String(r.ph), String(r.tds), String(r.t...
1260             row.forEach((v, i) => pdf.text(v.substring(0, 12), 14 + i * colW, y));
1261             y += 5;
1262         }
1263     }
1264
1265     pdf.save(filename);
1266 } catch (err) {
1267     setError(getApiErrorMessage(err));
1268 }
1269 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, readings]);
1270
1271 const fetchPeriodAggregation = useCallback(async (): Promise<AggregationResponse> => {
1272     if (periodType === "day") return fetchDaily(selectedDate);
1273     if (periodType === "week") return fetchWeekly(selectedYear, selectedWeek);
1274     return fetchMonthly(selectedYear, selectedMonth);
1275 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth]);
1276
1277 const handleDownloadReadingsCsv = useCallback(async () => {
1278     try {
1279         const fileBase = getPeriodFileBase("sensor-readings", periodType, selectedDate, selected...
1280         const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
1281             periodType, selectedDate, selectedYear, selectedWeek, selectedMonth,
1282         );
1283         const [agg, exportReadings] = await Promise.all([
1284             fetchPeriodAggregation(),
1285             fetchReadings({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }),
1286         ]);
1287         const summary = buildAggregationSummaryRows(agg, agg.period);
1288         const dataRows = sensorReadingExportRows(exportReadings);
1289         downloadCsvFile(`${fileBase}.csv`, [...summary, [], ["Sensor readings"], ...dataRows]);
1290     } catch (err) {
1291         setError(getApiErrorMessage(err));
1292     }

```

```
1293     }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, fetchPeriodAggregat...
1294
1295     const handleDownloadReadingsExcel = useCallback(async () => {
1296       try {
1297         const fileBase = getPeriodFileBase("sensor-readings", periodType, selectedDate, selected...
1298         const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
1299           periodType, selectedDate, selectedYear, selectedWeek, selectedMonth,
1300         );
1301         const [agg, exportReadings] = await Promise.all([
1302           fetchPeriodAggregation(),
1303           fetchReadings({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }),
1304         ]);
1305         const summary = buildAggregationSummaryRows(agg, agg.period);
1306         const dataRows = sensorReadingExportRows(exportReadings);
1307         downloadExcelFile(`${fileBase}.xlsx`, [
1308           { name: "Summary", rows: summary },
1309           { name: "Sensor readings", rows: dataRows },
1310         ]);
1311       } catch (err) {
1312         setError(getApiErrorMessage(err));
1313       }
1314     }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, fetchPeriodAggregat...
1315
1316     const handleDownloadPredictionsCsv = useCallback(async () => {
1317       try {
1318         const fileBase = getPeriodFileBase("predictions", periodType, selectedDate, selectedYear...
1319         const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
1320           periodType, selectedDate, selectedYear, selectedWeek, selectedMonth,
1321         );
1322         const [agg, exportPredictions, exportReadings] = await Promise.all([
1323           fetchPeriodAggregation(),
1324           fetchPredictions({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT...
1325           fetchReadings({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }),
1326         ]);
1327         const readingById = new Map(exportReadings.map((r) => [r.id, r]));
1328         const summary = buildAggregationSummaryRows(agg, agg.period);
1329         const dataRows = predictionExportRows(exportPredictions, readingById);
1330         downloadCsvFile(`${fileBase}.csv`, [...summary, [], ["Predictions"], ...dataRows]);
1331       } catch (err) {
1332         setError(getApiErrorMessage(err));
1333       }
1334     }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, fetchPeriodAggregat...
1335
1336     const handleDownloadPredictionsExcel = useCallback(async () => {
1337       try {
1338         const fileBase = getPeriodFileBase("predictions", periodType, selectedDate, selectedYear...
1339         const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
1340           periodType, selectedDate, selectedYear, selectedWeek, selectedMonth,
1341         );
1342         const [agg, exportPredictions, exportReadings] = await Promise.all([
1343           fetchPeriodAggregation(),
1344           fetchPredictions({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT...
1345           fetchReadings({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }),
1346         ]);
1347         const readingById = new Map(exportReadings.map((r) => [r.id, r]));
1348         const summary = buildAggregationSummaryRows(agg, agg.period);
1349         const dataRows = predictionExportRows(exportPredictions, readingById);
1350         downloadExcelFile(`${fileBase}.xlsx`, [
1351           { name: "Summary", rows: summary },
1352           { name: "Predictions", rows: dataRows },
1353         ]);
1354       } catch (err) {
1355         setError(getApiErrorMessage(err));
1356       }
1357     }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, fetchPeriodAggregat...
1358
1359     const buildDefectRows = useCallback(async () => {
1360       const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
```

```

1361     periodType, selectedDate, selectedYear, selectedWeek, selectedMonth,
1362   );
1363   const [exportPredictions, exportReadings] = await Promise.all([
1364     fetchPredictions({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }...
1365     fetchReadings({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }),
1366   ]);
1367   const readingById = new Map(exportReadings.map((r) => [r.id, r]));
1368   const headers = ["ID", "Date", "Time", "pH", "Sugar %", "Citric %", "Ascorbic %", "Issue"];
1369   const rows: string[][] = [headers];
1370   for (const p of exportPredictions.filter((x) => x.authenticity_status === "adulterated")) {
1371     const r = readingById.get(p.reading);
1372     const ph = r?.ph ?? 0;
1373     const sugar = p.predicted_sugar;
1374     const citric = p.predicted_citric;
1375     const ascorbic = p.predicted_ascorbic;
1376     const issues: string[] = [];
1377     if (ph < TYPICAL_RANGES.ph.min) issues.push(`pH low (${ph.toFixed(2)})`);
1378     else if (ph > TYPICAL_RANGES.ph.max) issues.push(`pH high (${ph.toFixed(2)})`);
1379     if (sugar < TYPICAL_RANGES.sugar.min) issues.push(`Sugar low (${sugar.toFixed(2)}%)`);
1380     else if (sugar > TYPICAL_RANGES.sugar.max) issues.push(`Sugar high (${sugar.toFixed(2)}%)`);
1381     if (citric < TYPICAL_RANGES.citric.min) issues.push(`Citric low (${citric.toFixed(4)}%)`);
1382     else if (citric > TYPICAL_RANGES.citric.max) issues.push(`Citric high (${citric.toFixed(4)}%)`);
1383     if (ascorbic < TYPICAL_RANGES.ascorbic.min) issues.push(`Ascorbic low (${ascorbic.toFixed(2)}%)`);
1384     else if (ascorbic > TYPICAL_RANGES.ascorbic.max) issues.push(`Ascorbic high (${ascorbic.toFixed(2)}%)`);
1385     const dt = new Date(p.timestamp);
1386     rows.push([
1387       String(p.id),
1388       format(dt, "yyyy-MM-dd"),
1389       format(dt, "HH:mm"),
1390       r?.ph != null ? String(r.ph) : "",
1391       String(sugar),
1392       String(citric),
1393       String(ascorbic),
1394       issues.length > 0 ? issues.join("; ") : "Out of range",
1395     ]);
1396   }
1397   return rows;
1398 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth]);
1399
1400 const handleDownloadDefectsCsv = useCallback(async () => {
1401   try {
1402     const fileBase = getPeriodFileBase("defects", periodType, selectedDate, selectedYear, se...
1403     const dataRows = await buildDefectRows();
1404     downloadCsvFile(`${fileBase}.csv`, [
1405       ["Coconut Water Quality Report – Defects"],
1406       ["Period", periodType === "day" ? selectedDate : periodType === "week" ? `${selectedYe...
1407       [],
1408       ...dataRows,
1409     ]);
1410   } catch (err) {
1411     setError(getApiErrorMessage(err));
1412   }
1413 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, buildDefectRows]);
1414
1415 const handleDownloadDefectsExcel = useCallback(async () => {
1416   try {
1417     const fileBase = getPeriodFileBase("defects", periodType, selectedDate, selectedYear, se...
1418     const dataRows = await buildDefectRows();
1419     downloadExcelFile(`${fileBase}.xlsx`, [{ name: "Defects", rows: dataRows }]);
1420   } catch (err) {
1421     setError(getApiErrorMessage(err));
1422   }
1423 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, buildDefectRows]);
1424
1425 const handleDownloadPredictionsPDF = useCallback(async () => {
1426   try {
1427     let filename: string;
1428     if (periodType === "day") {

```

```

1429     filename = `predictions-${selectedDate}.pdf`;
1430   } else if (periodType === "week") {
1431     filename = `predictions-${selectedYear}-W${selectedWeek}.pdf`;
1432   } else {
1433     filename = `predictions-${selectedYear}-${String(selectedMonth).padStart(2, "0")}.pdf`;
1434   }
1435
1436   const { dateFrom: exportFrom, dateTo: exportTo } = resolvePeriodRange(
1437     periodType,
1438     selectedDate,
1439     selectedYear,
1440     selectedWeek,
1441     selectedMonth,
1442   );
1443   const [exportPredictions, exportReadings] = await Promise.all([
1444     fetchPredictions({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT...
1445     fetchReadings({ date_from: exportFrom, date_to: exportTo, limit: EXPORT_FETCH_LIMIT }),
1446   ]);
1447
1448   const pdf = new jsPDF("portrait", "mm", "a4");
1449   const pageW = pdf.internal.pageSize.getWidth();
1450   let y = 15;
1451
1452   pdf.setFontSize(16);
1453   const title = "Coconut Water Quality Report";
1454   const titleX = (pageW - pdf.getTextWidth(title)) / 2;
1455   pdf.text(title, titleX, y);
1456   y += 10;
1457
1458   pdf.setFontSize(10);
1459   pdf.text(`Period: ${periodType} – ${periodType === "day" ? selectedDate : periodType ===...
1460   y += 6;
1461   pdf.text(`Total readings exported: ${exportPredictions.length}`, 14, y);
1462   y += 10;
1463
1464   if (exportPredictions.length > 0) {
1465     const readingById = new Map(exportReadings.map((r) => [r.id, r]));
1466     const colW = (pageW - 28) / 10;
1467     const headers = ["ID", "Reading", "Date", "Time", "pH", "Sugar %", "Citric %", "Ascorb...
1468     pdf.setFont("helvetica", "bold");
1469     headers.forEach((h, i) => pdf.text(h, 14 + i * colW, y));
1470     pdf.setFont("helvetica", "normal");
1471     y += 6;
1472     for (const p of exportPredictions) {
1473       if (y > 270) {
1474         pdf.addPage();
1475         y = 15;
1476       }
1477       const r = readingById.get(p.reading);
1478       const status = p.authenticity_status === "authentic" ? "Authentic" : "Adulterated";
1479       const dt = new Date(p.timestamp);
1480       const dateStr = format(dt, "yyyy-MM-dd");
1481       const timeStr = format(dt, "HH:mm");
1482       const row = [
1483         String(p.id),
1484         String(p.reading),
1485         dateStr,
1486         timeStr,
1487         r?.ph !== null ? String(r.ph) : "-",
1488         String(p.predicted_sugar),
1489         String(p.predicted_citric),
1490         String(p.predicted_ascorbic),
1491         status,
1492         confidencePercent(p.confidence),
1493       ];
1494       row.forEach((v, i) => pdf.text(v.substring(0, 8), 14 + i * colW, y));
1495       y += 5;
1496     }

```

```

1497     } else {
1498         pdf.text("No predictions for this period.", 14, y);
1499     }
1500
1501     pdf.save(filename);
1502 } catch (err) {
1503     setError(getApiErrorMessage(err));
1504 }
1505 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth]);
1506
1507 const handleDownloadDefectsPDF = useCallback(async () => {
1508     try {
1509         const defects = predictions
1510             .filter((p) => p.authenticity_status === "adulterated")
1511             .map((p) => {
1512                 const r = readings.find((x) => x.id === p.reading);
1513                 const ph = r?.ph ?? 0;
1514                 const sugar = p.predicted_sugar;
1515                 const citric = p.predicted_citric;
1516                 const ascorbic = p.predicted_ascorbic;
1517                 const issues: string[] = [];
1518                 if (ph < TYPICAL_RANGES.ph.min) issues.push(`pH low (${ph.toFixed(2)})`);
1519                 else if (ph > TYPICAL_RANGES.ph.max) issues.push(`pH high (${ph.toFixed(2)})`);
1520                 if (sugar < TYPICAL_RANGES.sugar.min) issues.push(`Sugar low (${sugar.toFixed(2)})%`);
1521                 else if (sugar > TYPICAL_RANGES.sugar.max) issues.push(`Sugar high (${sugar.toFixed(2)})%`);
1522                 if (citric < TYPICAL_RANGES.citric.min) issues.push(`Citric low (${citric.toFixed(4)})`);
1523                 else if (citric > TYPICAL_RANGES.citric.max) issues.push(`Citric high (${citric.toFixed(4)})`);
1524                 if (ascorbic < TYPICAL_RANGES.ascorbic.min) issues.push(`Ascorbic low (${ascorbic.toFixed(2)})`);
1525                 else if (ascorbic > TYPICAL_RANGES.ascorbic.max) issues.push(`Ascorbic high (${ascorbic.toFixed(2)})`);
1526                 return {
1527                     id: p.id,
1528                     date: format(new Date(p.timestamp), "yyyy-MM-dd"),
1529                     time: format(new Date(p.timestamp), "HH:mm"),
1530                     ph: r?.ph ?? "-",
1531                     sugar,
1532                     citric,
1533                     ascorbic,
1534                     issue: issues.length > 0 ? issues.join("; ") : "Out of range",
1535                 };
1536             });
1537
1538         let filename: string;
1539         if (periodType === "day") {
1540             filename = `defects-${selectedDate}.pdf`;
1541         } else if (periodType === "week") {
1542             filename = `defects-${selectedYear}-W${selectedWeek}.pdf`;
1543         } else {
1544             filename = `defects-${selectedYear}-${String(selectedMonth).padStart(2, "0")}.pdf`;
1545         }
1546
1547         const pdf = new jsPDF("portrait", "mm", "a4");
1548         const pageW = pdf.internal.pageSize.getWidth();
1549         let y = 15;
1550
1551         pdf.setFontSize(16);
1552         const title = "Coconut Water Quality Report – Defects";
1553         const titleX = (pageW - pdf.getTextWidth(title)) / 2;
1554         pdf.text(title, titleX, y);
1555         y += 10;
1556
1557         pdf.setFontSize(10);
1558         pdf.text(`Period: ${periodType === "day" ? selectedDate : periodType === "week" ? `sel...` : ""}`);
1559         y += 10;
1560
1561         if (defects.length > 0) {
1562             const colW = (pageW - 28) / 8;
1563             const headers = ["ID", "Date", "Time", "pH", "Sugar %", "Citric %", "Ascorbic %", "Iss..."];
1564             pdf.setFont("helvetica", "bold");

```

```

1565     headers.forEach((h, i) => pdf.text(h, 14 + i * colW, y));
1566     pdf.setFont("helvetica", "normal");
1567     y += 6;
1568     for (const d of defects) {
1569         if (y > 270) {
1570             pdf.addPage();
1571             y = 15;
1572         }
1573         const row = [
1574             String(d.id),
1575             d.date,
1576             d.time,
1577             String(d.ph),
1578             String(d.sugar),
1579             String(d.citric),
1580             String(d.ascorbic),
1581             d.issue.substring(0, 30),
1582         ];
1583         row.forEach((v, i) => pdf.text(v.substring(0, 10), 14 + i * colW, y));
1584         y += 5;
1585     }
1586 } else {
1587     pdf.text("No low-quality defects in this period.", 14, y);
1588 }
1589
1590 pdf.save(filename);
1591 } catch (err) {
1592     setError(getApiErrorMessage(err));
1593 }
1594 }, [periodType, selectedDate, selectedYear, selectedWeek, selectedMonth, predictions, readin...
1595
1596 useEffect(() => {
1597     loadAll();
1598     // eslint-disable-next-line react-hooks/exhaustive-deps
1599 }, []);
1600
1601 const formatTs = (ts: string) => {
1602     try {
1603         return format(new Date(ts), "yyyy-MM-dd HH:mm");
1604     } catch {
1605         return ts;
1606     }
1607 };
1608
1609 /** Join sensor readings with ML predictions for correlation / trend graphs. */
1610 const joinedGraphRows = useMemo(() => {
1611     const readingById = new Map(readings.map((r) => [r.id, r]));
1612     return [...predictions]
1613         .sort((a, b) => new Date(a.timestamp).getTime() - new Date(b.timestamp).getTime())
1614         .map((p) => {
1615             const r = readingById.get(p.reading);
1616             if (!r) return null;
1617             return {
1618                 time: format(new Date(p.timestamp), "HH:mm:ss"),
1619                 date: format(new Date(p.timestamp), "yyyy-MM-dd"),
1620                 pH: r.ph,
1621                 TDS: r.tds,
1622                 Temperature: r.temperature,
1623                 Turbidity: r.turbidity,
1624                 Sugar: p.predicted_sugar,
1625                 Citric: p.predicted_citric,
1626                 Ascorbic: p.predicted_ascorbic,
1627                 status: p.authenticity_status,
1628             };
1629         })
1630         .filter((row): row is NonNullable<typeof row> => row != null);
1631     }, [readings, predictions]);
1632

```

```
1633     const sensorGraphData = useMemo(  
1634       () =>  
1635         [...readings]  
1636           .sort((a, b) => new Date(a.timestamp).getTime() - new Date(b.timestamp).getTime())  
1637           .slice(-100)  
1638           .map((r) => ({  
1639             time: format(new Date(r.timestamp), "HH:mm:ss"),  
1640             pH: r.pH,  
1641             TDS: r.tds,  
1642             Temperature: r.temperature,  
1643             Turbidity: r.turbidity,  
1644           })),  
1645       [readings],  
1646     );  
1647  
1648     const predictionGraphData = useMemo(  
1649       () =>  
1650         joinedGraphRows.slice(-100).map((row) => ({  
1651           time: row.time,  
1652           Sugar: row.Sugar,  
1653           Citric: row.Citric,  
1654           Ascorbic: row.Ascorbic,  
1655         })),  
1656       [joinedGraphRows],  
1657     );  
1658  
1659     const authenticityGraphData = useMemo(  
1660       () => [  
1661         {  
1662           name: "Samples",  
1663           Authentic: predictions.filter((p) => p.authenticity_status === "authentic").length,  
1664           Adulterated: predictions.filter((p) => p.authenticity_status === "adulterated").length,  
1665         },  
1666       ],  
1667       [predictions],  
1668     );  
1669  
1670     const pearsonR = (xs: number[], ys: number[]): number | null => {  
1671       const n = Math.min(xs.length, ys.length);  
1672       if (n < 2) return null;  
1673       const x = xs.slice(0, n);  
1674       const y = ys.slice(0, n);  
1675       const meanX = x.reduce((s, v) => s + v, 0) / n;  
1676       const meanY = y.reduce((s, v) => s + v, 0) / n;  
1677       let num = 0;  
1678       let denX = 0;  
1679       let denY = 0;  
1680       for (let i = 0; i < n; i++) {  
1681         const dx = x[i] - meanX;  
1682         const dy = y[i] - meanY;  
1683         num += dx * dy;  
1684         denX += dx * dx;  
1685         denY += dy * dy;  
1686       }  
1687       const den = Math.sqrt(denX * denY);  
1688       if (den === 0) return null;  
1689       return num / den;  
1690     };  
1691  
1692     const correlationCharts = useMemo(() => {  
1693       const rows = joinedGraphRows;  
1694       const make = (  
1695         title: string,  
1696         xKey: "TDS" | "pH" | "Temperature" | "Turbidity",  
1697         yKey: "Sugar" | "Citric" | "Ascorbic",  
1698         xLabel: string,  
1699         yLabel: string,  
1700         color: string,
```

```

1701     ) => {
1702       const data = rows.map((r) => ({
1703         x: r[xKey],
1704         y: r[yKey],
1705         status: r.status,
1706       }));
1707       const r = pearsonR(
1708         data.map((d) => d.x),
1709         data.map((d) => d.y),
1710       );
1711       return { title, xLabel, yLabel, color, data, r };
1712     };
1713     return [
1714       make("Sugar % vs TDS", "TDS", "Sugar", "TDS (ppm)", "Sugar %", "#27AE60"),
1715       make("Citric acid % vs pH", "pH", "Citric", "pH", "Citric acid %", "#F5A623"),
1716       make("Ascorbic acid % vs pH", "pH", "Ascorbic", "pH", "Ascorbic acid %", "#14B8A6"),
1717       make("Sugar % vs pH", "pH", "Sugar", "pH", "Sugar %", "#4A90E2"),
1718       make("Citric acid % vs TDS", "TDS", "Citric", "TDS (ppm)", "Citric acid %", "#E67E22"),
1719       make("Ascorbic acid % vs TDS", "TDS", "Ascorbic", "TDS (ppm)", "Ascorbic acid %", "#8B5C...
1720     ];
1721   }, [joinedGraphRows]);
1722
1723   const location = useLocation();
1724   const panelIndex = useMemo(() => {
1725     const p = location.pathname;
1726     if (p === "/admin/readings") return 0;
1727     if (p === "/admin/predictions") return 1;
1728     if (p === "/admin/logs") return 2;
1729     if (p === "/admin/graphs") return 3;
1730     return -1; // Overview
1731   }, [location.pathname]);
1732
1733   const isOverview = panelIndex === -1;
1734
1735   return (
1736     <div className="admin-dashboard-panels admin-dashboard-full">
1737       {error && (
1738         <div className="admin-error-banner" role="alert">
1739           <p style={{ margin: 0 }}>{error}</p>
1740           <button type="button" onClick={() => setError(null)} aria-label="Dismiss">x</button>
1741         </div>
1742       )}
1743       {isOverview && (
1744         <div className="admin-report-overview-wrap">
1745           <SensorPredictPanel
1746             currentSnapshot={homeDemo?.sensorSnapshot ?? liveSnapshot}
1747             bufferedCount={esp32Buffer.count}
1748             requiredCount={esp32Buffer.required}
1749             waiting={!homeDemo && !liveSnapshot}
1750           />
1751           <ProductionReportOverview
1752             readings={readings}
1753             liveReadings={liveReadings}
1754             activeReadingIndex={sessionComplete ? 0 : esp32Buffer.count + 1}
1755             bufferedCount={esp32Buffer.count}
1756             requiredCount={esp32Buffer.required}
1757             homeDemo={homeDemo}
1758             robotStage={robotStage}
1759             completedVerdict={completedVerdict}
1760           />
1761         </div>
1762       )}
1763     </section>
1764     <section
1765       className={`dashboard-card admin-panel-pane admin-readings-panel ${panelIndex === 0 ? ...
1766       style={{ marginBottom: "1.5rem", overflowX: "auto", display: panelIndex === 0 ? "block...
1767       aria-hidden={panelIndex !== 0}
1768     >

```

```

1769     <h2 className="dashboard-section-title">Sensor readings</h2>
1770     <PeriodSelectorBar
1771       periodType={periodType}
1772       setPeriodType={setPeriodType}
1773       selectedDate={selectedDate}
1774       setSelectedDate={setSelectedDate}
1775       selectedYear={selectedYear}
1776       setSelectedYear={setSelectedYear}
1777       selectedWeek={selectedWeek}
1778       setSelectedWeek={setSelectedWeek}
1779       selectedMonth={selectedMonth}
1780       setSelectedMonth={setSelectedMonth}
1781       onApply={handleApplyPeriod}
1782       onDownloadPdf={handleDownloadPDF}
1783       onDownloadCsv={handleDownloadReadingsCsv}
1784       onDownloadExcel={handleDownloadReadingsExcel}
1785       loading={loadingReadings || loadingPredictions || loadingLogs}
1786       variant="light"
1787     />
1788     {loadingReadings ? (
1789       <p className="admin-readings-loading">Loading...</p>
1790     ) : (
1791       <div className="admin-readings-table-wrap">
1792         <table className="admin-readings-table">
1793           <thead>
1794             <tr>
1795               <th>ID</th>
1796               <th>Date</th>
1797               <th>Time</th>
1798               <th>pH</th>
1799               <th>TDS</th>
1800               <th>Temp</th>
1801               <th>Turbidity</th>
1802             </tr>
1803           </thead>
1804           <tbody>
1805             {readings.length === 0 ? (
1806               <tr><td colspan={7}>No readings.</td></tr>
1807             ) : (
1808               readings.map((r) => {
1809                 const d = (() => {
1810                   try {
1811                     const dt = new Date(r.timestamp);
1812                     return { date: format(dt, "yyyy-MM-dd"), time: format(dt, "HH:mm") };
1813                   } catch {
1814                     return { date: "-", time: "-" };
1815                   }
1816                 })();
1817                 return (
1818                   <tr key={r.id}>
1819                     <td>{r.id}</td>
1820                     <td>{d.date}</td>
1821                     <td>{d.time}</td>
1822                     <td>{r.ph}</td>
1823                     <td>{r.tds}</td>
1824                     <td>{r.temperature}</td>
1825                     <td>{r.turbidity}</td>
1826                   </tr>
1827                 );
1828               })
1829             )}
1830           </tbody>
1831         </table>
1832       </div>
1833     )}
1834   </section>
1835
1836   <section

```

```

1837     className={`dashboard-card admin-panel-pane admin-predictions-panel ${panelIndex === 1...
1838     style={{ marginBottom: "1.5rem", overflowX: "auto", display: panelIndex === 1 ? "block...
1839     aria-hidden={panelIndex !== 1}
1840   >
1841   <h2 className="dashboard-section-title">Predictions</h2>
1842   <PeriodSelectorBar
1843     periodType={periodType}
1844     setPeriodType={setPeriodType}
1845     selectedDate={selectedDate}
1846     setSelectedDate={setSelectedDate}
1847     selectedYear={selectedYear}
1848     setSelectedYear={setSelectedYear}
1849     selectedWeek={selectedWeek}
1850     setSelectedWeek={setSelectedWeek}
1851     selectedMonth={selectedMonth}
1852     setSelectedMonth={setSelectedMonth}
1853     onApply={handleApplyPeriod}
1854     onDownloadPdf={handleDownloadPredictionsPDF}
1855     onDownloadCsv={handleDownloadPredictionsCsv}
1856     onDownloadExcel={handleDownloadPredictionsExcel}
1857     loading={loadingReadings || loadingPredictions || loadingLogs}
1858     variant="light"
1859   />
1860   {loadingPredictions ? (
1861     <p className="admin-predictions-loading">Loading...</p>
1862   ) : (
1863     <div className="admin-predictions-table-wrap">
1864       <table className="admin-predictions-table">
1865         <thead>
1866           <tr>
1867             <th>ID</th>
1868             <th>Reading</th>
1869             <th>Date</th>
1870             <th>Time</th>
1871             <th>pH</th>
1872             <th>Sugar %</th>
1873             <th>Citric %</th>
1874             <th>Ascorbic %</th>
1875             <th>Status</th>
1876             <th>Confidence</th>
1877           </tr>
1878         </thead>
1879         <tbody>
1880           {predictions.length === 0 ? (
1881             <tr><td colspan={10}>No predictions.</td></tr>
1882           ) : (
1883             predictions.map((p) => {
1884               const reading = readings.find((r) => r.id === p.reading);
1885               const statusLabel = p.authenticity_status === "authentic" ? "Authentic" ...
1886               const dt = (() => {
1887                 try {
1888                   const d = new Date(p.timestamp);
1889                   return { date: format(d, "yyyy-MM-dd"), time: format(d, "HH:mm") };
1890                 } catch {
1891                   return { date: "-", time: "-" };
1892                 }
1893               })();
1894               return (
1895                 <tr key={p.id}>
1896                   <td>{p.id}</td>
1897                   <td>{p.reading}</td>
1898                   <td>{dt.date}</td>
1899                   <td>{dt.time}</td>
1900                   <td>{reading?.ph ?? "-"}</td>
1901                   <td>{p.predicted_sugar}</td>
1902                   <td>{p.predicted_citric}</td>
1903                   <td>{p.predicted_ascorbic}</td>
1904                   <td style={{ color: p.authenticity_status === "adulterated" ? "#f871...

```

```

1905                 {statusLabel}
1906             </td>
1907             <td>{formatConfidence(p.confidence)}</td>
1908         </tr>
1909     );
1910 })
1911 })
1912 </tbody>
1913 </table>
1914 </div>
1915 })
1916 </section>
1917
1918 {/* Defects: low-quality predictions only */}
1919 <section
1920     className={`dashboard-card admin-panel-pane admin-defects-panel ${panelIndex === 2 ? "...
1921     style={{ overflowX: "auto", display: panelIndex === 2 ? "block" : "none" }}
1922     aria-hidden={panelIndex !== 2}
1923 >
1924     <h2 className="dashboard-section-title">Defects</h2>
1925     <PeriodSelectorBar
1926         periodType={periodType}
1927         setPeriodType={setPeriodType}
1928         selectedDate={selectedDate}
1929         setSelectedDate={setSelectedDate}
1930         selectedYear={selectedYear}
1931         setSelectedYear={setSelectedYear}
1932         selectedWeek={selectedWeek}
1933         setSelectedWeek={setSelectedWeek}
1934         selectedMonth={selectedMonth}
1935         setSelectedMonth={setSelectedMonth}
1936         onApply={handleApplyPeriod}
1937         onDownloadPdf={handleDownloadDefectsPDF}
1938         onDownloadCsv={handleDownloadDefectsCsv}
1939         onDownloadExcel={handleDownloadDefectsExcel}
1940         loading={loadingReadings || loadingPredictions || loadingLogs}
1941         variant="light"
1942     />
1943     {loadingPredictions ? (
1944         <p className="admin-defects-loading">Loading...</p>
1945     ) : (
1946         <div className="admin-defects-table-wrap">
1947             <table className="admin-defects-table">
1948                 <thead>
1949                     <tr>
1950                         <th>ID</th>
1951                         <th>Date</th>
1952                         <th>Time</th>
1953                         <th>pH</th>
1954                         <th>Sugar %</th>
1955                         <th>Citric %</th>
1956                         <th>Ascorbic %</th>
1957                         <th>Issue</th>
1958                     </tr>
1959                 </thead>
1960                 <tbody>
1961                     {(() => {
1962                         const defects = predictions
1963                         .filter((p) => p.authenticity_status === "adulterated")
1964                         .map((p) => {
1965                             const r = readings.find((x) => x.id === p.reading);
1966                             const ph = r?.ph ?? 0;
1967                             const sugar = p.predicted_sugar;
1968                             const citric = p.predicted_citric;
1969                             const ascorbic = p.predicted_ascorbic;
1970                             const issues: string[] = [];
1971                             if (ph < TYPICAL_RANGES.ph.min) issues.push(`pH low (${ph.toFixed(2)})`);
1972                             else if (ph > TYPICAL_RANGES.ph.max) issues.push(`pH high (${ph.toFixed(...

```

```

1973         if (sugar < TYPICAL_RANGES.sugar.min) issues.push(`Sugar low (${sugar.to...
1974         else if (sugar > TYPICAL_RANGES.sugar.max) issues.push(`Sugar high (${su...
1975         if (citric < TYPICAL_RANGES.citric.min) issues.push(`Citric low (${citri...
1976         else if (citric > TYPICAL_RANGES.citric.max) issues.push(`Citric high ($...
1977         if (ascorbic < TYPICAL_RANGES.ascorbic.min) issues.push(`Ascorbic low ($...
1978         else if (ascorbic > TYPICAL_RANGES.ascorbic.max) issues.push(`Ascorbic h...
1979         return {
1980             id: p.id,
1981             date: format(new Date(p.timestamp), "yyyy-MM-dd"),
1982             time: format(new Date(p.timestamp), "HH:mm"),
1983             ph: r?.ph ?? "-",
1984             sugar: p.predicted_sugar,
1985             citric: p.predicted_citric,
1986             ascorbic: p.predicted_ascorbic,
1987             issue: issues.length > 0 ? issues.join("; ") : "Out of range",
1988         };
1989     });
1990     if (defects.length === 0) {
1991         return (
1992             <tr><td colSpan={8}>No low-quality defects in this period.</td></tr>
1993         );
1994     }
1995     return defects.map((d) => (
1996         <tr key={d.id}>
1997             <td>{d.id}</td>
1998             <td>{d.date}</td>
1999             <td>{d.time}</td>
2000             <td>{d.ph}</td>
2001             <td>{d.sugar}</td>
2002             <td>{d.citric}</td>
2003             <td>{d.ascorbic}</td>
2004             <td className="admin-defects-issue">{d.issue}</td>
2005         </tr>
2006     ));
2007     })()}
2008 </tbody>
2009 </table>
2010 </div>
2011 )}
2012 </section>
2013
2014 <section
2015     className={`dashboard-card admin-panel-pane admin-graphs-panel ${panelIndex === 3 ? "a...
2016     style={{ display: panelIndex === 3 ? "block" : "none" }}
2017     aria-hidden={panelIndex !== 3}
2018 >
2019     <h2 className="dashboard-section-title">Graph</h2>
2020     <div className="admin-report-period-bar admin-report-period-bar--light">
2021         <label>
2022             Date:{" "}
2023             <input
2024                 type="date"
2025                 value={selectedDate}
2026                 onClick={(event) => event.currentTarget.showPicker?.()}
2027                 onFocus={(event) => event.currentTarget.showPicker?.()}
2028                 onKeyDown={(event) => {
2029                     if (event.key !== "Tab") event.preventDefault();
2030                 }}
2031                 onPaste={(event) => event.preventDefault()}
2032                 onChange={(event) => handleGraphDateChange(event.target.value)}
2033                 aria-label="Select graph date from calendar"
2034                 style={{ cursor: "pointer" }}
2035             />
2036         </label>
2037         {(loadingReadings || loadingPredictions) && <span>Loading...</span>}
2038     </div>
2039     <p style={{ marginTop: "0.75rem", color: "#e2e8f0" }}>
2040         Trends and correlation graphs for {dateFrom} to {dateTo}

```

```

2041     {joinedGraphRows.length > 0 ? ` (${joinedGraphRows.length} paired samples)` : ""}.
2042   </p>
2043
2044   {sensorGraphData.length === 0 && joinedGraphRows.length === 0 ? (
2045     <p style={{ color: "#e2e8f0" }}>No graph data available for this period. Select a da...
2046   ) : (
2047     <div className="admin-graphs-stack">
2048       {correlationCharts.map((chart) => (
2049         <div key={chart.title} className="admin-report-panel admin-report-chart-panel">
2050           <h3>
2051             {chart.title}
2052             {chart.r !== null ? (
2053               <span style={{ marginLeft: "0.5rem", fontWeight: 500, color: "#64748b", fo...
2054               (r = {chart.r.toFixed(3)})
2055             </span>
2056             ) : null}
2057           </h3>
2058           <div className="admin-chart-container">
2059             {chart.data.length === 0 ? (
2060               <p style={{ padding: "1rem", color: "#64748b" }}>No paired samples in this...
2061             ) : (
2062               <ResponsiveContainer width="100%" height={300}>
2063                 <ScatterChart margin={{ top: 12, right: 16, bottom: 12, left: 8 }}>
2064                   <CartesianGrid strokeDasharray="3 3" />
2065                   <XAxis
2066                     type="number"
2067                     dataKey="x"
2068                     name={chart.xLabel}
2069                     label={{ value: chart.xLabel, position: "insideBottom", offset: -2 }}
2070                   />
2071                   <YAxis
2072                     type="number"
2073                     dataKey="y"
2074                     name={chart.yLabel}
2075                     label={{ value: chart.yLabel, angle: -90, position: "insideLeft" }}
2076                   />
2077                   <Tooltip
2078                     cursor={{ strokeDasharray: "3 3" }}
2079                     formatter={(value: number, name: string) => [
2080                       typeof value === "number" ? value.toFixed(4) : value,
2081                       name === "x" ? chart.xLabel : chart.yLabel,
2082                     ]}
2083                   />
2084                   <Scatter name={chart.title} data={chart.data} fill={chart.color} />
2085                 </ScatterChart>
2086               </ResponsiveContainer>
2087             )}
2088           </div>
2089         </div>
2090       )]}
2091
2092   <div className="admin-report-panel admin-report-chart-panel">
2093     <h3>pH and Temperature (trend)</h3>
2094     <div className="admin-chart-container">
2095       <ResponsiveContainer width="100%" height={300}>
2096         <LineChart data={sensorGraphData}>
2097           <CartesianGrid strokeDasharray="3 3" />
2098           <XAxis dataKey="time" />
2099           <YAxis />
2100           <Tooltip />
2101           <Legend />
2102           <Line type="monotone" dataKey="pH" stroke="#4A90E2" strokeWidth={2} dot={f...
2103           <Line type="monotone" dataKey="Temperature" stroke="#F5A623" strokeWidth={...
2104         </LineChart>
2105       </ResponsiveContainer>
2106     </div>
2107   </div>
2108

```

```

2109     <div className="admin-report-panel admin-report-chart-panel">
2110       <h3>TDS and Turbidity (trend)</h3>
2111       <div className="admin-chart-container">
2112         <ResponsiveContainer width="100%" height={300}>
2113           <LineChart data={sensorGraphData}>
2114             <CartesianGrid strokeDasharray="3 3" />
2115             <XAxis dataKey="time" />
2116             <YAxis />
2117             <Tooltip />
2118             <Legend />
2119             <Line type="monotone" dataKey="TDS" stroke="#27AE60" strokeWidth={2} dot={...
2120             <Line type="monotone" dataKey="Turbidity" stroke="#8B5CF6" strokeWidth={2}...
2121           </LineChart>
2122         </ResponsiveContainer>
2123       </div>
2124     </div>
2125
2126     <div className="admin-report-panel admin-report-chart-panel">
2127       <h3>Predicted Composition (trend)</h3>
2128       <div className="admin-chart-container">
2129         <ResponsiveContainer width="100%" height={300}>
2130           <LineChart data={predictionGraphData}>
2131             <CartesianGrid strokeDasharray="3 3" />
2132             <XAxis dataKey="time" />
2133             <YAxis yAxisId="sugar" />
2134             <YAxis yAxisId="acids" orientation="right" />
2135             <Tooltip />
2136             <Legend />
2137             <Line yAxisId="sugar" type="monotone" dataKey="Sugar" stroke="#27AE60" str...
2138             <Line yAxisId="acids" type="monotone" dataKey="Citric" stroke="#F5A623" st...
2139             <Line yAxisId="acids" type="monotone" dataKey="Ascorbic" stroke="#14B8A6" ...
2140           </LineChart>
2141         </ResponsiveContainer>
2142       </div>
2143     </div>
2144
2145     <div className="admin-report-panel admin-report-chart-panel">
2146       <h3>Authenticity Results</h3>
2147       <div className="admin-chart-container">
2148         <ResponsiveContainer width="100%" height={300}>
2149           <BarChart data={authenticityGraphData}>
2150             <CartesianGrid strokeDasharray="3 3" />
2151             <XAxis dataKey="name" />
2152             <YAxis allowDecimals={false} />
2153             <Tooltip />
2154             <Legend />
2155             <Bar dataKey="Authentic" fill="#22C55E" />
2156             <Bar dataKey="Adulterated" fill="#EF4444" />
2157           </BarChart>
2158         </ResponsiveContainer>
2159       </div>
2160     </div>
2161   </div>
2162   )}
2163 </section>
2164 </div>
2165 );
2166 }
2167

```